CE/CS/SE 3354 · Software Engineering · Fall 2026

Project Milestone P2

Analysis and High-Level Design

Study Room Reservation System for UTD

Team	Team 07, section .010
Document version	2.0
Date	11/8/2026
Repository URL	https://github.com/matthewlow262-jpg/Study-Room-Reservation-System
Git tag for this milestone	p2 (points at the commit that contains this exact PDF)
Submitted on Canvas by	Ernie Hsiao

Team member	NetID	GitHub username
Matthew Low	Mjl230002	matthewlow262-jpg
Arjun Sharma	axs220320	sharmararjun-droid
Ernie Hsiao	exh230014	ErnieHsiao
Andrew Artiaga	axa220205	AndyA64
Artyom Gevorgyan	axg200155	ptdg
Param Kathiravan	Pxk230069	paramkathir

0 Revision History, Contribution Record and Changes Since P1

All three tables below are graded. The contribution record carries the individual 20% of this milestone and must agree with your repository history.

Version	Date	Author	Summary of change
1.0	11/6/2026	All team members	P1 baseline (copy your last P1 row)
1.1	11/8/2026	All team members	P2 baseline: analysis and high-level design added
1.2	11/8/2026	Matthew Low	Finished Section 4
1.3	11/8/2026	Andrew Artiaga	Finished Section 6.3, Section 6.4, GitHub diagram uploads
1.4	11/8/2026	Param Kathiravan	Finished Section 8 and 9
1.5	11/8/2026	Arjun Sharma	Filled in sections 6.1, 6.2, 6.5, 6.6, and 7.2
1.6	11/8/2026	Matthew Low	Filled in Section 7.1
1.7	11/8/2026	Arjun Sharma	Updated Section 10.1
1.8	11/8/2026	Matthew Low	Updated Section 2.4, 2.6, 3.1, 3.3, 3.5 and 9.1
2.0	11/8/2026	All team members	Finished all sections

0.1 Individual contribution record (P2)

Team member	Sections and artifacts this person produced for P2	Evidence in the repository (commit hashes, PR numbers)	Reviews given (PR numbers)
Matthew Low	Section 4.1-4.3 Section 7.1 Sections 2.4, 2.6, 3.1, 3.3, 3.5 and 9.1. edits	Eb15a8063b685d720b880ad6b2428711aba33c8a (#6) c6b8ca06f3ba1fa467db7be6d314988784e1fafc (#11) eb9c6e64070ae8d202ec38c4802f7a5919848ac4 (#12)	#7, #8, #13

Team member	Sections and artifacts this person produced for P2	Evidence in the repository (commit hashes, PR numbers)	Reviews given (PR numbers)
Arjun Sharma	Section 6.1, 6.2 Section 6.5 Section 6.6 Section 7.2 Section 10.1 edits	1932ae5980d612fd23ea1a233be7ef097cdfdc68 (#3) D9dedff03cfb2e9ef2f5b308590e68aacd1ff8e4 (#5) 11f53e806a2dd4d0e4155912791706d3911ddcff (#7) Afd7f68ecef716bc6024443b91ca3ce402b6caa9 (#9) 18f5b2b26a2498a76641bedd06a7d98bad5a1f50 (#13)	#1, #12
Ernie Hsiao	Section 5, Section 10	0732e2db6a9ecf8ad38da142e0aad6cd6911451c (#4), 85480a3a76b1ba8de72380e584517cfe50c2019c (#13)	#5, #2, #6, #10
Andrew Artiaga	Sections 1-3 from P1 copied to P2, Section 6.3, Section 6.4, GitHub diagram uploads & contribution record	e48c95d1691b5d2bbfdba4d9ce8b178a9fdb4ae8 (#2) 4bf9b89d4ed01dae64e1fd5c1818df5d3c50acfb (#8)	#3, #4, #9
Artyom Gevorgyan	Section 5.2	4c9cd8d2656c77bf94be5ed37f3a7042d38fbbda (#10)	#11
Param Kathiravan	Section 8 — Architecture Spike S-01; Section 9.1 — Extended Traceability Matrix; Section 9.2 — Gap Analysis	8fac0da3b6f49bed1f460e19e5d5fef1eee30145 (#1)	#11

0.2 Changes since the P1 baseline

Change ID	What changed (IDs affected)	Why (P1 feedback, new finding, spike result)	Knock-on effect	Agreed on
CH-01	UC-01 condensed steps	Steps were too simple and added unnecessary clutter	5.2.1 is easier to read	Agreed Nov. 7
CH-02	US-01 split last step into 2	Too complex to map in the scenario table	5.2.1 is easier to read	Agreed Nov. 7
CH-03				

1 to 3 Introduction, Overall Description and Requirements (carried forward)

1 Introduction

1.1 Purpose of this document and of the system

This document is going to define the initial project proposal as well as the software requirements for the Study Room Reservation System for the University of Texas at Dallas. This document will describe the purpose, scope, stakeholders, constraints, requirements, and use cases. It is intended for the project team, instructor, and other stakeholders involved in reviewing, developing, and evaluating the project.

The Study Room Reservation System is going to give students a convenient way to find available study rooms on campus and reserve them for individual or group study. This system will allow students to search for rooms based on availability, time, and capacity. It will also let authorized personnel manage the room information.

1.2 Scope

In scope — we will build this	Out of scope — we will not, and why
Display a list of all study rooms with their locations and vacancies	Interacting with any physical locks and doors that require comet cards, keys, etc. Because this would require us to have access to external hardware and infrastructure that is only available to certain staff in UTD.
Allow students to search for rooms based off date, time, and capacity	Tracking real-time room occupancy through cameras or sensors because we are focusing on reserving rooms rather than monitoring the physical occupancy of the room
Display available time slots for a room	Integrating with UTD’s current campus scheduling system or software because we do not have access to those systems
Allow students to make reservations for available time slots	Sending SMS notifications because we would require access to personal information from the students and that information is not currently required for the start of this project

In scope — we will build this	Out of scope — we will not, and why
Prevent overlapping reservations for the same room and time	Implementing UTD Duo authentication because that would require access to UTD's authentication system
Allow authorized administrators to manage the study room listings	Processing payments or charging students for room reservations. This feature is not necessary for the purpose of this system
Send email notifications regarding the status of the room reservation (confirmation/cancellation).	Integrate with external university services that have not been approved or made available to the team

Why the exclusions were excluded:

If we did include the exclusions, then the project would be much more difficult to create and to implement. The exclusions would require that we have access to external permissions and tools that go beyond the intended purpose of the program. By focusing on the aspects of the project that are in scope, we will be able to lower the complexity of the project and allow it to be used for its intended purpose.

1.3 Definitions, acronyms and abbreviations

Term or abbreviation	Meaning as used in this project
UTD	University of Texas at Dallas
Study room	A designated space for students to reserve for self-study or group study
Reservation	A booking that assigns a study room to whichever student booked it
System	An operating system designed and coded to allow students to make reservations
Availability	The status of a study room that shows if it is reserved or if it is free

1.4 References

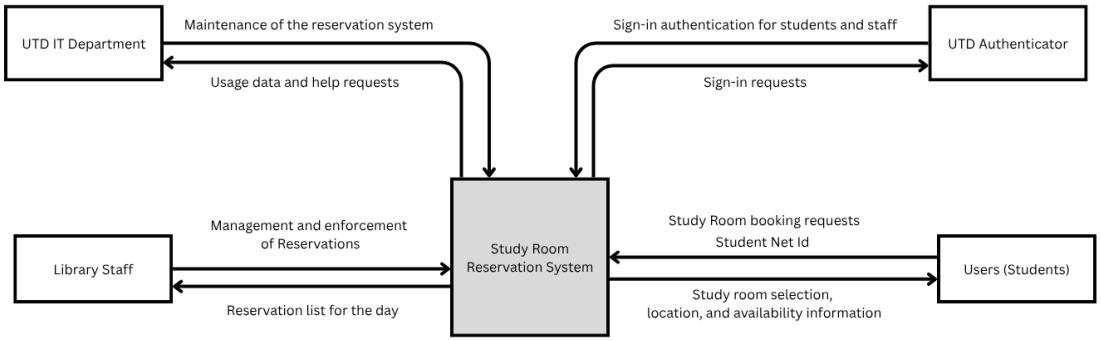
[R1] CE 3354 Fall 2026 course syllabus and P1 project template

[R2] UTD library policies; https://library.utdallas.edu/about-the-library/library-policies/

[R3] UTD policies on information security or privacy; https://policy.utdallas.edu/utdbp3096

2 Overall Description

2.1 Product perspective and context



Why the boundary sits here: Inside the box is everything we build: the room catalogue, availability search, reservation rules, admin tools for room listings, and the logic that triggers emails. Outside is everything we depend on but don't control: UTD's identity records, its email service, and the physical doors, locks and Comet Card readers. Only students and library staff use the system directly.

2.2 Stakeholders and how you reached them

Stakeholder	Role in relation to the system	What they need from it	How you learned this
Students	Users that use the system	- Reservation of rooms - Cancellation of reservations - Confirmation of reservation - Way to view availability	Based on our experiences as students, finding a study room in the library can be difficult at times, such as during midterms. These needs are currently based on our observations and assumptions.
Library Staff	Manage the study room area	- Room management capabilities, both physical and remote	We assume library staff are responsible for managing study room availability and resolving room-related issues. These assumptions have not

Stakeholder	Role in relation to the system	What they need from it	How you learned this
			yet been confirmed through interviews.
UTD IT Department	Create and maintain the system	- System security and user authentication integration - Database management capabilities - Compliance with UTD privacy policies	Based on our understanding of the technical requirements for a university reservation system. The specific authentication, infrastructure, and privacy requirements still need to be verified with UTD IT staff.
UTD Office of Budget and Finance	Fund the servers and system	- Proof that the system is being used - Proof that the system is helpful	We assume that funding decisionmakers would need evidence of system usefulness and operating costs before supporting continued funding.

2.3 User characteristics

Users: Students at UTD

They know: Where the study rooms are, study room hours, size of their party

They don't know: Which rooms are available, when rooms are available, how big the rooms are

User constraints: UTD ID

2.4 Constraints

The registration service must be hosted by UTD's servers. The data used to track students is personal and should remain on the UTD servers.

The system must comply with UTD’s rules and policies regarding information security and acceptable use when handling university and student data.

The students must leave the study rooms at least 30 minutes before the library closes. This is to give the library staff time to clean up.

2.5 Assumptions and dependencies

#	Assumption or dependency	What breaks if it does not hold
A1	We depend on the library staff to manage the study rooms.	There will be no one to update or enforce the reservations.
A2	We assume the students and library staff have access to the internet.	They will not be able to place or change a reservation.
A3	An email delivery method will be available if email notifications are in scope	Confirmation and cancellation emails may not be deliverable

2.6 Feasibility summary

Feasibility gate	Verdict	Evidence behind the verdict
Technical	Potentially feasible	The S-01 prototype tested 20 simultaneous reservation attempts across 100 trials. Exactly one reservation succeeded in every trial, supporting the feasibility of preventing any overlapping reservations. Other features like room searching, email notifications, or staff management need further testing
Operational	Potentially feasible	Students can make reservations for available study rooms and the system will deal with conflicting reservations and if needed, the staff can manage the reservations as well. The team still needs to test the workflow with students and staff in the library
Schedule	Feasible	We have identified core functions that the system needs to be successful and have a milestone plan. Evidence of schedule feasibility will include assigned tasks to owners, completed milestones, recorded progress, etc. This way the schedule remains achievable
Economic / resource	Potentially feasible	The proposed scope does not include tools like paid reservation processing or integration with whole university systems. These conflicts could prove to be difficult because even if our project works it only remains potentially feasible because we cannot control things like integrating our system with the universities software.

3 Specific Requirements

3.1 Functional requirements

ID	Requirement — one “shall” sentence	Priority	Source	Acceptance criterion
FR-01	The system shall display the list of study rooms in its room listing	MUST	Students	A tester can confirm this by verifying that every room in the test dataset appears in the list
FR-02	The system shall display the location of each listed study room	MUST	Students	Verify that each of the listed rooms displays its location
FR-03	The system shall display the capacity of each listed study room	MUST	Students	Verify that each listed room displays its capacity
FR-04	The system shall allow students to search for rooms by reservation date.	MUST	Students	Search using a date and ensure that only rooms with availability are returned
FR-05	The system shall allow students to search for rooms by reservation time	MUST	Students	Search using a time range and make sure that the results match the availability of the rooms
FR-06	The system shall allow students to filter rooms by minimum capacity	SHOULD	Students	Enter a capacity and verify that each result either meets or exceeds it
FR-07	The system shall display available time slots for a selected room	MUST	Students	Select a room and verify that the time slots match the rooms reservations
FR-08	The system shall allow a student to reserve an available time slot	MUST	Students	Submit a valid reservation and verify that the reservation is recorded
FR-09	The system shall prevent a reservation from overlapping with an existing reservation in the same room	MUST	Students	Attempt an overlapping booking and verify that it is rejected
FR-10	The system shall allow a student to cancel an existing reservation that they submitted	MUST	Students	Cancel a reservation and verify that its status is updated and the slot is now free
FR-11	The system shall display reservations made by the student requesting a reservation	COULD	Students	Verify that the student can see their own reservation records without seeing other students' reservation records

ID	Requirement — one “shall” sentence	Priority	Source	Acceptance criterion
FR-12	The system shall allow authorized library staff to update the location and capacity of existing study room listings.	MUST	Library Staff	Log in as an authorized administrator, modify a room's location and capacity, save the changes, and verify that the updated information appears in the room listing.
FR-13	The system shall allow authorized library staff to view study room reservation usage data for a selected date range.	SHOULD	Library Staff	Select a date range as an authorized administrator and verify that the system displays reservation usage data for that range.
FR-14	The system shall send an email notification to the student after a reservation is made successfully	MUST	Students	Create a valid reservation and verify the confirmation email was sent to the student
FR-15	The system shall send an email notification to the students after a reservation is canceled	MUST	Students	Cancel an existing reservation and verify that the cancellation email is sent to the student

3.2 Quality requirements

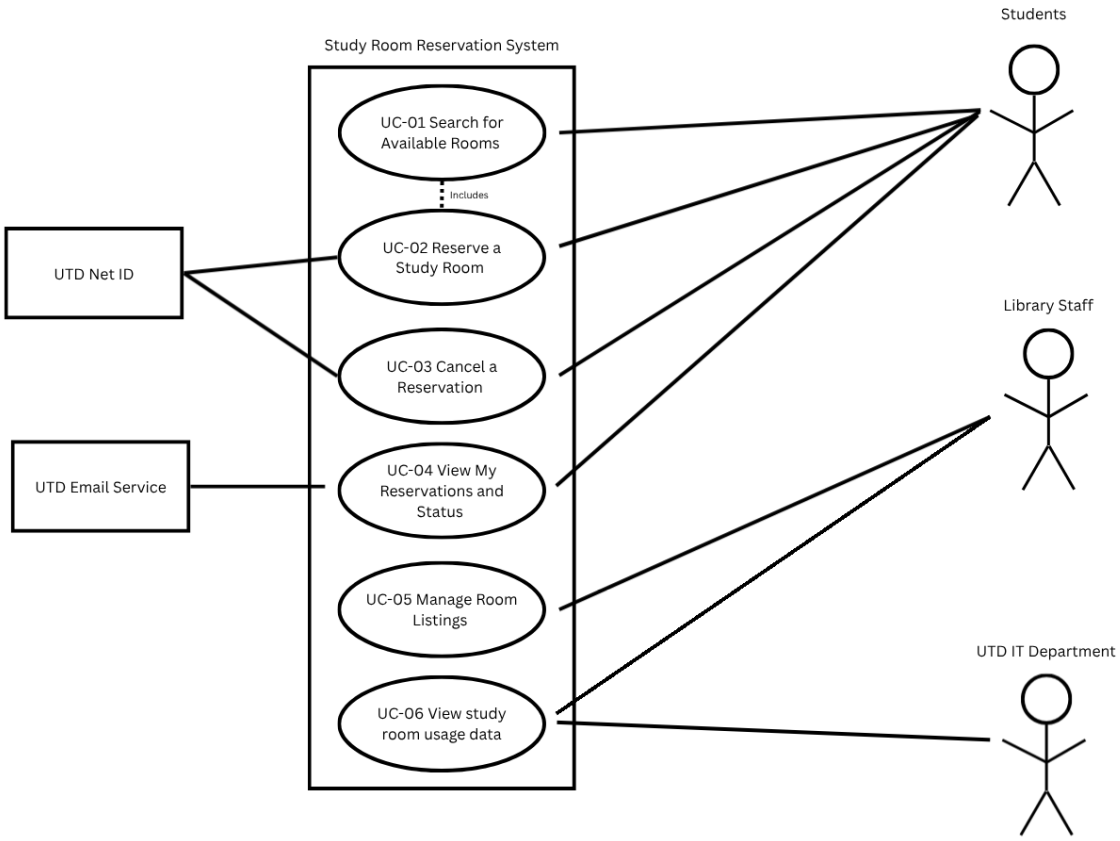
ID	Requirement — with threshold, condition and method	Priority	Source	How it will be verified
QR-01	95% of room searches (FR-04 and FR-05) will return results within 2 seconds for up to 25 concurrent users and 50 rooms.	MUST	Students	Load script of 200 searches run with 25 simultaneous users; the 95th-percentile response time is read from the script log.
QR-02	When 20 users attempt to reserve the same room and time slot at the same moment, exactly one reservation shall be created and 19 shall be rejected, in each of 100 repeated runs.	MUST	Students	Automated concurrency tests run 100 times; the database is checked after each run for a single confirmed reservation.
QR-03	At least 80 percent of first-time student testers (minimum 5 testers) shall find a room and complete a reservation in under 3 minutes without help, on a phone-size screen 360 pixels wide.	SHOULD	Students	Moderated usability test with a stopwatch and a task script; completion times and unaided

ID	Requirement — with threshold, condition and method	Priority	Source	How it will be verified
				successes are recorded.
QR-04	The system shall be available at least 99 percent of the time during library opening hours, measured over any 14-day period.	SHOULD	Students	A monitoring service requests the home page every minute during library opening hours; availability is computed from its log.

3.3 Constraint requirements

ID	Constraint — one sentence	Source	Why it cannot be negotiated
CON-01	The system shall be hosted on UTD-operated servers, and student personal data shall not be stored anywhere else.	UTD IT	Student identity data is personal, and UTD IT is responsible for keeping it on UTD servers.
CON-02	The system shall comply with UTD's rules and policies, including those on student privacy detailed in UTD’s homepage privacy section.	UTD IT	A university rule is not the teams to waive.

3.4 Initial use cases



Actors: Student and Library staff (administrator); supporting actor: UTD email service. System boundary as in 2.1.

UC-01 Search for Available Rooms

UC-02 Reserve a Study Room

UC-03 Cancel a Reservation

UC-04 View My Reservations and Status

UC-05 Manage Room Listings

UC-06 View study room usage data

Signing in is a precondition of UC-02 to UC-05, not a use case on its own: it is a step toward a goal, not a goal.

Field	Content
Identifier and name	UC-02 Reserve a Study Room
Primary actor	Student
Supporting actors	None confirmed; email service
Preconditions — what must be true before it starts	The student is identified by the system, and an available study room and time slot exist
Trigger — what starts it	The student selects an available room and chooses to reserve it
Main success scenario — numbered steps	1. The student selects an available study room 2. The system displays available time slots 3. The student selects a time slot 4. The student submits the reservation request 5. The system checks for conflicting reservations. 6. The system records the reservation 7. The system displays confirmation to the student
Alternative and exception flows	**3a**. If the selected time slot is unavailable, the system rejects the request and asks the student to select another slot. **4a**. If the student's identity cannot be confirmed, the system prevents the reservation. **6a**. If the reservation cannot be saved, the system reports the failure without confirming the booking
Postconditions — what is true when it succeeds	The reservation is recorded, and the selected room and time slot are no longer available for overlapping reservations.
Requirements realized	FR- 07, FR-08, FR-09

Expanded use case

Field	Content
Identifier and name	UC-05 Manage Room Listings
Primary actor	Library Staff (Administrator)
Supporting actors	None confirmed
Preconditions — what must be true before it starts	The library staff member is signed in with administrator permissions. The system has access to the current room listings.

Trigger — what starts it	The library staff member chooses to manage the study room listings.
Main success scenario — numbered steps	1. The administrator opens the room management section. 2. The system displays the current study room listings. 3. The administrator selects a room to update. 4. The system displays the room's current information. 5. The administrator modifies the room's information, such as location or capacity. 6. The administrator submits the changes. 7. The system validates the updated information. 8. The system saves the changes and displays confirmation.
Alternative and exception flows	**1a.** If the administrator enters invalid room for information, the system rejects the changes and requests corrections. **7a.** If the changes would conflict with existing reservations, the system rejects the update and informs the administrator. **7b.** If the administrator does not have the required permissions, the system denies access. **8a.** If the changes cannot be saved, the system displays an error and keeps the previous room information unchanged.
Postconditions — what is true when it succeeds	The updated room location or capacity is saved in the system and displayed in the room listings. Existing reservations remain unchanged.
Requirements realized	FR-01, FR-02, FR-03, FR-12

3.5 Requirements to use case traceability

The extended traceability matrix is provided in section 9.1 and it connects the functional requirements, quality requirements, and constraints to the use cases, analysis classes, architectural components, and supporting evidence.

4 Use-Case Analysis

Chapter 7, first half. Here you take the use cases from section 3.4 and write the most important of them out turn by turn, still with the technology switched off.

4.1 Use cases selected for analysis

Use case	Name	Why it was selected	Non-trivial steps
UC-02	UC-02 Reserve a study room	One of the 2 we specified at P1	Selecting a room and time slot, checking for existing reservations, validating the reservation request, recording the reservation, displaying confirmation
UC-05	UC-05 Manage Room Listings	One of the 2 we specified at P1	Selecting a room, modifying room information, validate changes, checking for conflicts, updating saved information
UC-01	UC-01 Search for available rooms	This was selected because being able to find an available study room is one of the main purposes of the system. Students need to be able to search for rooms without any errors	Entering search criteria, checking for availability, filtering rooms by capacity, displaying results

4.2 Expanded use cases

UC-01 [Search for Available rooms]

Primary actor	Student
Supporting actors and external systems	None
TUCBW (this use case begins when)	The student wants to find an available study room for a specific date, time and minimum room capacity
TUCEW (this use case ends when)	The system displays the study rooms that match the student's search criteria
Preconditions	Study room listings are available in the system The student has access to the study room reservation system
Postconditions	Matching study rooms and their availability are displayed No reservation is created with this use case
Traces to requirements	FR-01, FR-02, FR-03, FR-04, FR-05, FR-06, FR-07, QR-01

UC-02 [Reserve a study room]

Primary actor	Student
Supporting actors and external systems	UTD email service
TUCBW (this use case begins when)	The student selects an available study room and wants to reserve a specific date and slot
TUCEW (this use case ends when)	The system records the reservation, marks the time slot as unavailable, displays a reservation confirmation, and sends a confirmation email to the user
Preconditions	The student is identified by the system The student selects an available study room The selected reservation time does not conflict with other
Postconditions	The reservation is recorded by the system The selected time slot is now unavailable for other reservations A reservation confirmation is displayed and sent to the student
Traces to requirements	FR-07, FR-08, FR-09, QR-02, QR-03, CON-01, CON-02

UC-05 [Manage room listings]

Primary actor	Staff
Supporting actors and external systems	None

TUCBW (this use case begins when)	A library staff member wants to update the information of existing study rooms
TUCEW (this use case ends when)	The system successfully saves the updated room information and displays confirmation that the room listing was updated
Preconditions	The staff member has been identified by the system The staff member has administrative permissions The existing study room listings are available in the system
Postconditions	The updated room location or capacity is saved The updated information is shown in the room listings Existing reservations are unchanged unless needed to
Traces to requirements	FR-01, FR-02, FR-03, FR-12, CON-01, CON-02

UC-01 [Search for Available rooms]

Actor input / action	System response
	(0) The system displays the available room search options
(1) The student enters the desired reservation date and time	(2) System accepts the selected date and time
(3) The student enters the minimum room capacity needed	(4) The system accepts the minimum capacity
(5) The student submits the search request	(6) The system searches for rooms matching the date, time, and capacity.
	(7) The system displays matching rooms with their locations, capacities, and available time slots.

Alternate flow	At step	Condition	What happens	Ends or rejoins at
A1	7	The student enters incomplete or invalid search criteria	The system tells the student that the search criteria is wrong	Rejoins at step 1
A2	8	No rooms match the search criteria	The system informs the student that no rooms are available	Ends
A3	8	Some rooms do not meet the minimum capacity	The system excludes those rooms from the search results	Rejoins at step 9

UC-02 [Reserve a study room]

Actor input / action	System response
	(0) The system displays available study rooms and time slots
(1) The student selects a study room and an available time slot	(2) The system displays the selected room, date and time for review
(3) The student confirms the reservation request	(4) The system checks whether the selected time slot is available or not
	(5) The system records the reservation and marks the selected time slot as unavailable
(8) The student reviews the reservation confirmation	(6) The system displays a confirmation
	(7) The system sends a confirmation email to the student

Alternate flow	At step	Condition	What happens	Ends or rejoins at
A1	4	Another student has already reserved the selected time	The system rejects the request and tells the student that the time is not available	Ends
A2	5	The system cannot save the reservation	The system informs the student that the reservation can't be completed	Ends
A3	7	The confirmation email can't be sent	The system informs the student that the reservation was recorded but the confirmation email can't be sent	Ends

UC-05 [Manage room listings]

Actor input / action	System response
	(0) The system displays the room management options
(1) The staff member selects the option to manage room listings	(2) The system displays the current list of study rooms
(3) The staff member selects a room to update	(4) The system displays the selected rooms current location and capacity
(5) The staff member enters the updated room location or capacity	(6) The system accepts the updated information
(7) The staff member submits the changes	(8) The system validates the update room information and saves the changes.

Alternate flow	At step	Condition	What happens	Ends or rejoins at
A1	1	The staff member does not have administrator permissions	The system denies access to room management options	Ends
A2	8	Updated information is invalid or incomplete	The system informs the staff member that the information must be corrected	Rejoins at step 5
A3	8	The updated room information conflicts with existing reservations	The system informs the staff member that the changes cannot be completed because of the conflict	Ends
A4	9	The system cannot save the changes	The system informs the staff that the update failed and the room keeps its original information	Ends

4.3 Self-review log

Use case	Check from the Lesson 13 review checklist	Defect found	How it was fixed
UC-01	Verified that the use case has a clear trigger, preconditions, postconditions, and a complete sequence of actor-system interactions	The initial description did not clearly explain what happens when no rooms match the student's search criteria	Added an alternative flow explaining that the system informs the student when no matching rooms are available
UC-02	Checked that the use case shows a clear confirmation and explains what happens when something goes wrong	The initial description did not explicitly address what would happen if two students try and reserve the same room and time slot	Added an alternative flow explaining that the system rejects a reservation if another student has already reserved the spot
UC-05	Verified that the primary actor is clearly identified, preconditions are defined, and alternative flows cover invalid information	The initial description did not explain what happens if updated room information conflicts with existing reservations	Added an alternative flow explaining that the system rejects changes that conflict with existing reservations

5 Analysis Models

Chapter 7, second half. What has to exist inside the system for the use cases in section 4 to happen, and how those parts talk to each other.

5.1 Domain model

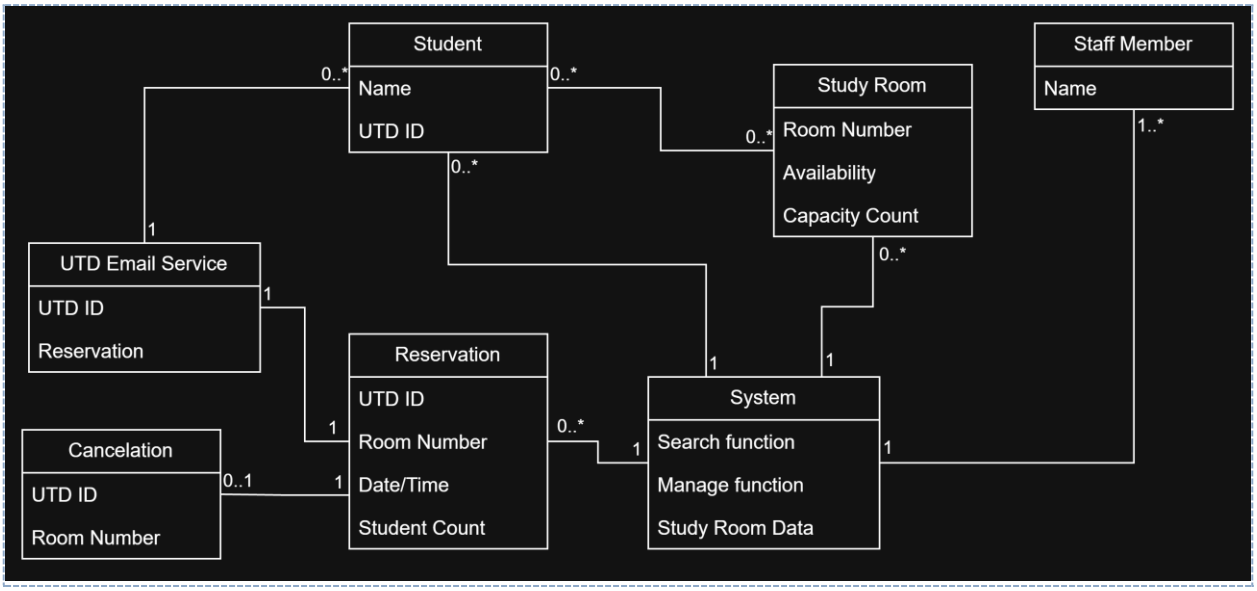


Figure 5.1 Domain model (conceptual classes, attributes, associations with multiplicities)

Class	What it represents (one line)	Key attributes	Associations (with multiplicity)
Student	Person making a reservation	Name, UTD ID	0..*
Staff Member	Person managing the reservations	Name	1..*
Study Room	Room being reserved by a student	Room Number, Availability, Capacity Count	0..*
Reservation	Details of a reservation by a student	UTD ID, Room Number, Date/Time, Student Count	0..*
System	System controlling the reservations	Search function, Manage function, Study Room Data	1
Cancelation	Canceled reservation	UTD ID, Room Number	0..1
UTD Email Service	Email service provided by UTD	UTD ID, Reservation	1

5.2 Object interaction models

5.2.1 UC-01 [Search for Available Rooms]

(a) Steps 1-7 require objects inside the system. The other steps only display values that already exist.

(b) Scenario table.

Step	Subject (who acts)	Subject action (what it does)	Other data / objects (what is used)	Object acted upon (who receives)
1	Student	Inputs the desired date and time	Date, time	System
2	System	Save date and time	Search bar	System
3	Student	Inputs the student count	Student count	System
4	System	Save that many students	Search bar	System
5	Student	Submits search request	Search criteria, available study rooms	System
6	System	Filter through to find available rooms	Search function, room number	Study Room
7	System	Return available study rooms		Student

(c) Analysis sequence diagram.

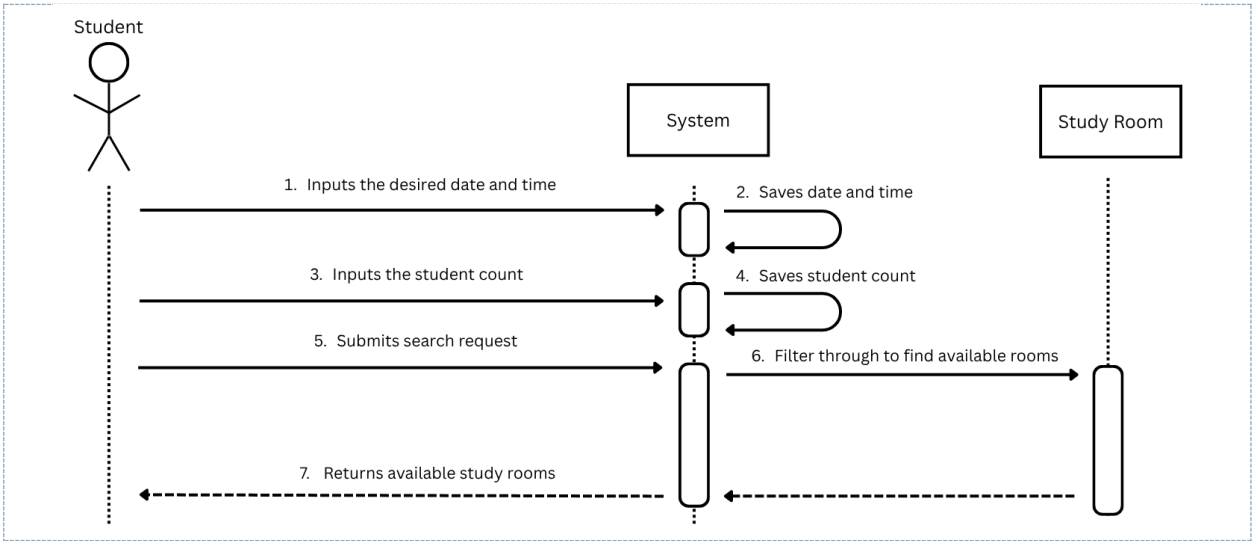


Figure 5.2.1 Analysis sequence diagram for UC-01

5.2.2 UC-05 [Manage Room Listings]

(a) Steps 1-8 require objects inside the system. The other steps only display values that already exist.

(b) Scenario table.

Step	Subject (who acts)	Subject action (what it does)	Other data / objects (what is used)	Object acted upon (who receives)
1	Staff member	Selects option to manage room listings	Manage function	System
2	System	Fetches the current list of study rooms	Reserved study rooms	Study Room
3	System	Returns list of study rooms	Study room list	Staff member
4	Staff member	Selects a room to update	Reserved study rooms	System
5	System	Fetches selected room data	Room number, availability, capacity count	Study Room
6	System	Returns selected room data	Room number, availability, capacity count	Staff member
7	Staff member	Updates room data	Room Number and/or Availability and/or Capacity Count	System
8	System	Accepts the information	Room Number and/or Availability and/or Capacity Count	System
9	Staff member	Submits the changes	Manage function	System
10	System	Validates and saves changes	Reservation	Study Room

(c) Analysis sequence diagram.

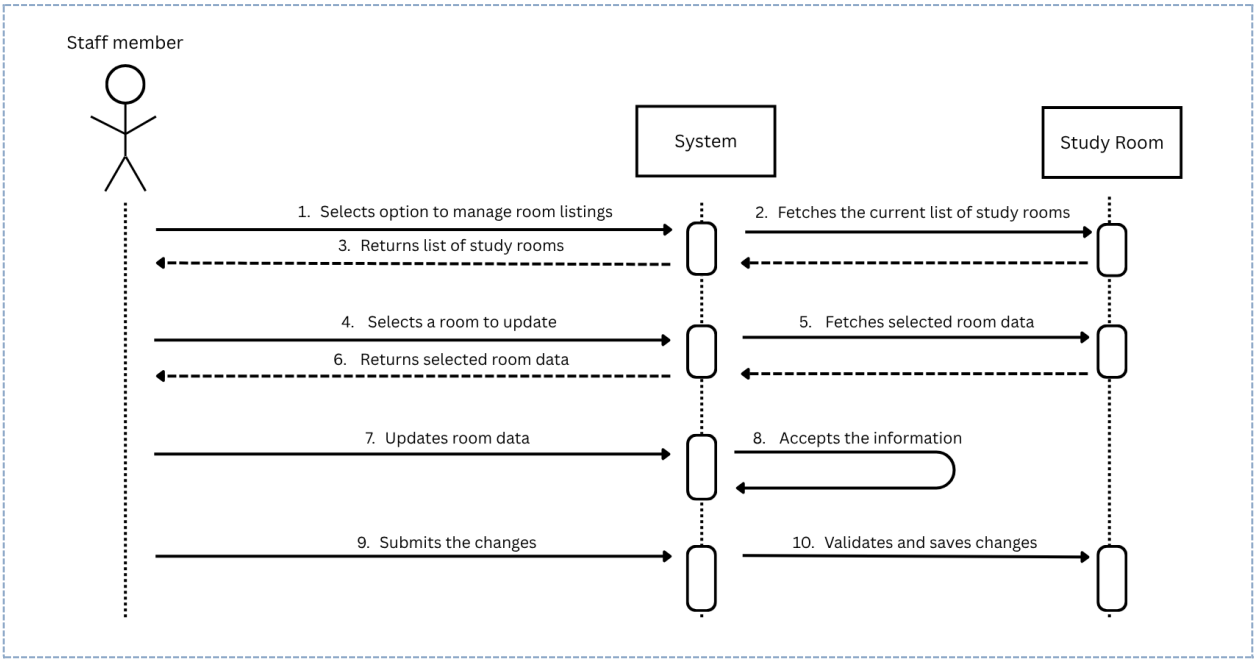


Figure 5.2.2 Analysis sequence diagram for UC-05

5.3 Consistency cross-check

Check	Result (pass, or what failed)	Fixed in
K1 Every item in an actor column (4.2) is an attribute in the domain model (5.1)	Pass	
K2 Every object on a sequence diagram is a domain class or a declared interaction object	Pass	
K3 Every message on a sequence diagram corresponds to exactly one scenario-table row	Pass	
K4 Every non-trivial step in 5.2 (a) has at least one scenario-table row	Pass	
K5 Every analyzed use case traces to at least one functional requirement	Pass	
K6 The last messages of each diagram make the use case postcondition true	Pass	

6 High-Level Design (Architecture)

Chapter 8. The rule now reverses: a high-level design that does not say where things run, what they are built from and what that choice cost has not been done.

6.1 Architectural drivers: quality-attribute scenarios

QAS	Quality attribute	Stimulus	Artifact	Response	Response measure	From
QAS-01	Performance	Up to 25 concurrent users submit room searches against a catalogue of 50 rooms	Search Controller, Room Catalogue, Reservation Repository	Return matching rooms and time slots without making the student wait excessively	At least 95% of 200 searches complete within 2 seconds	QR-01; UC-01
QAS-02	Concurrency	20 students attempt to reserve the same room and time slot simultaneously	Reservation Service and Reservation Database	Exactly one reservation is committed; all other requests are rejected without duplicate bookings	In each of the 100 repeated runs, 1 confirmed reservation and 19 rejected requests	QR-02; UC-02; R4
QAS-03	Usability	A first-time student uses a phone-size screen to find and reserve a room	Web UI and reservation workflow	The student completes the task without assistance using clear search and confirmation steps.	At least 80% of at least 5 first-time testers complete within 3 minutes on a 360-pixel-wide screen	QR-03; UC-01; UC-02

6.2 Architectural style and alternatives

Style considered	Drivers it serves	What it costs	Verdict
Layered three-tier web application (presentation, application/domain, data)	Clear separation of UI, reservation rules, catalogue/reporting logic, and persistence; supports QR-01/QR-02 and straightforward deployment.	Requires disciplined interfaces between layers; a single application deployment limits independent scaling.	Chosen
Microservices	Independent scaling and deployment of search, reservation, reporting, and notification functions.	More deployment, network, monitoring, and consistency complexity than this course project needs; cross-service reservation integrity is harder.	Rejected
Event-driven architecture	Loose coupling for notifications and asynchronous work.	Add broker/event delivery and eventual-consistency concerns; unnecessary for the small core reservation workflow.	Rejected

6.3 Component view

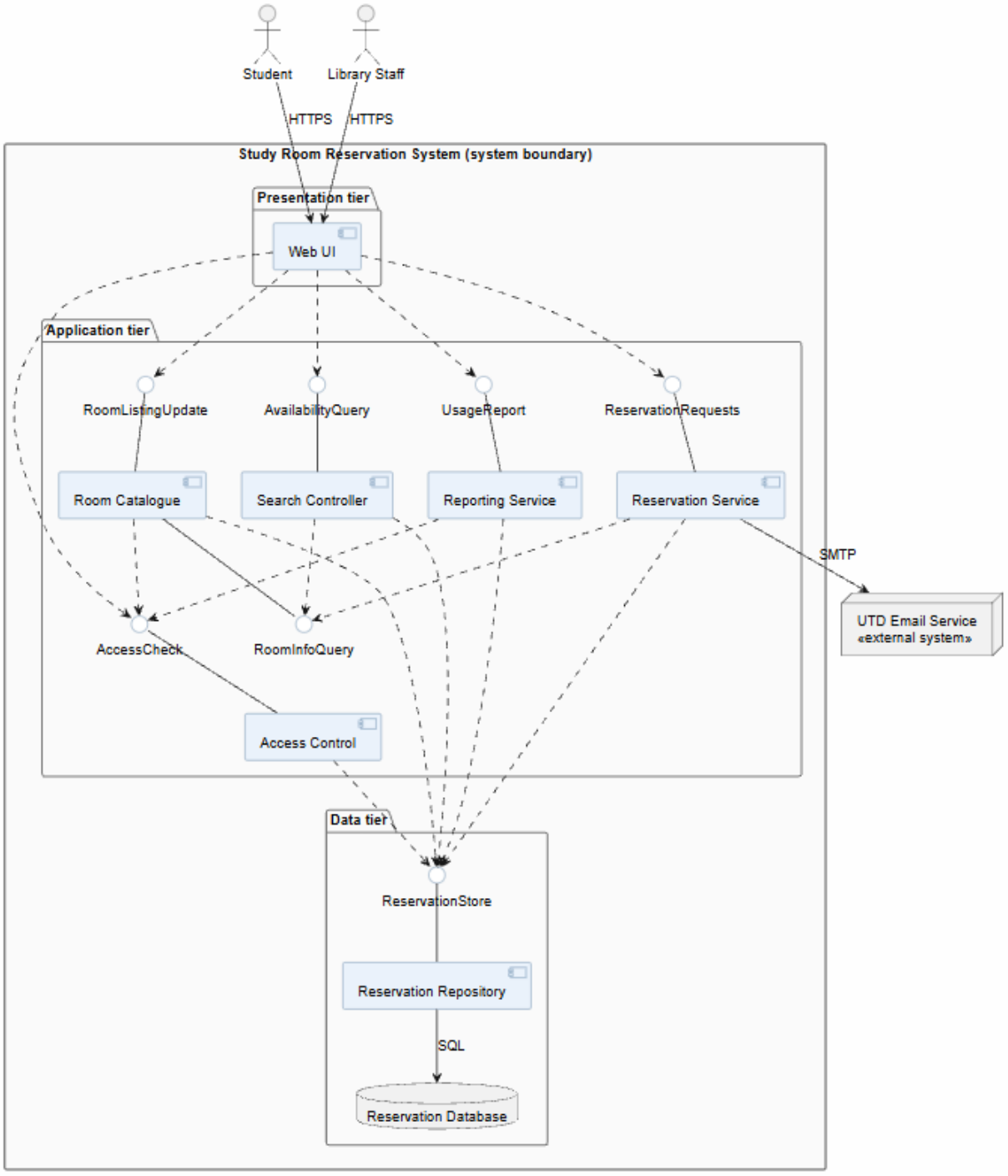


Figure 6.3 Component diagram: components, provided and required interfaces, external systems

Component	Responsibility (one sentence)	Provides	Requires	Answers to (UC / QAS / ADR)
Web UI	Phone-friendly pages for students and staff.	HTTPS pages	AccessCheck, AvailabilityQuery, ReservationRequests, RoomListingUpdate, UsageReport	UC-01–UC-06, QAS-03, ADR-01
Access Control	Signs users in and checks the staff role.	AccessCheck	ReservationStore	UC-02–UC-06 (sign-in precondition), UC-05
Search Controller	Finds rooms free for a date, time and capacity.	AvailabilityQuery	RoomInfoQuery, ReservationStore	UC-01, QAS-01
Room Catalogue	Keeps room details and applies staff edits that don't conflict with existing bookings.	RoomInfoQuery, RoomListingUpdate	AccessCheck, ReservationStore	UC-01, UC-05, QAS-01
Reservation Service	Validates, creates, cancels and lists reservations, and emails a confirmation.	ReservationRequests	RoomInfoQuery, ReservationStore, SMTP	UC-02–UC-04, QAS-02
Reporting Service	Summarizes room usage for a date range.	UsageReport	AccessCheck, ReservationStore	UC-06
Reservation Repository	Does all database access and commits each booking in a single transaction.	ReservationStore	SQL	QAS-01, QAS-02
Reservation Database	Stores rooms, reservations and users on UTD servers; a unique constraint is the final guard against double bookings.	SQL	none	QAS-02, ADR-01
UTD Email Service *(external)*	Delivers confirmation emails.	SMTP	none	UC-02 (supporting actor)

6.4 Deployment view

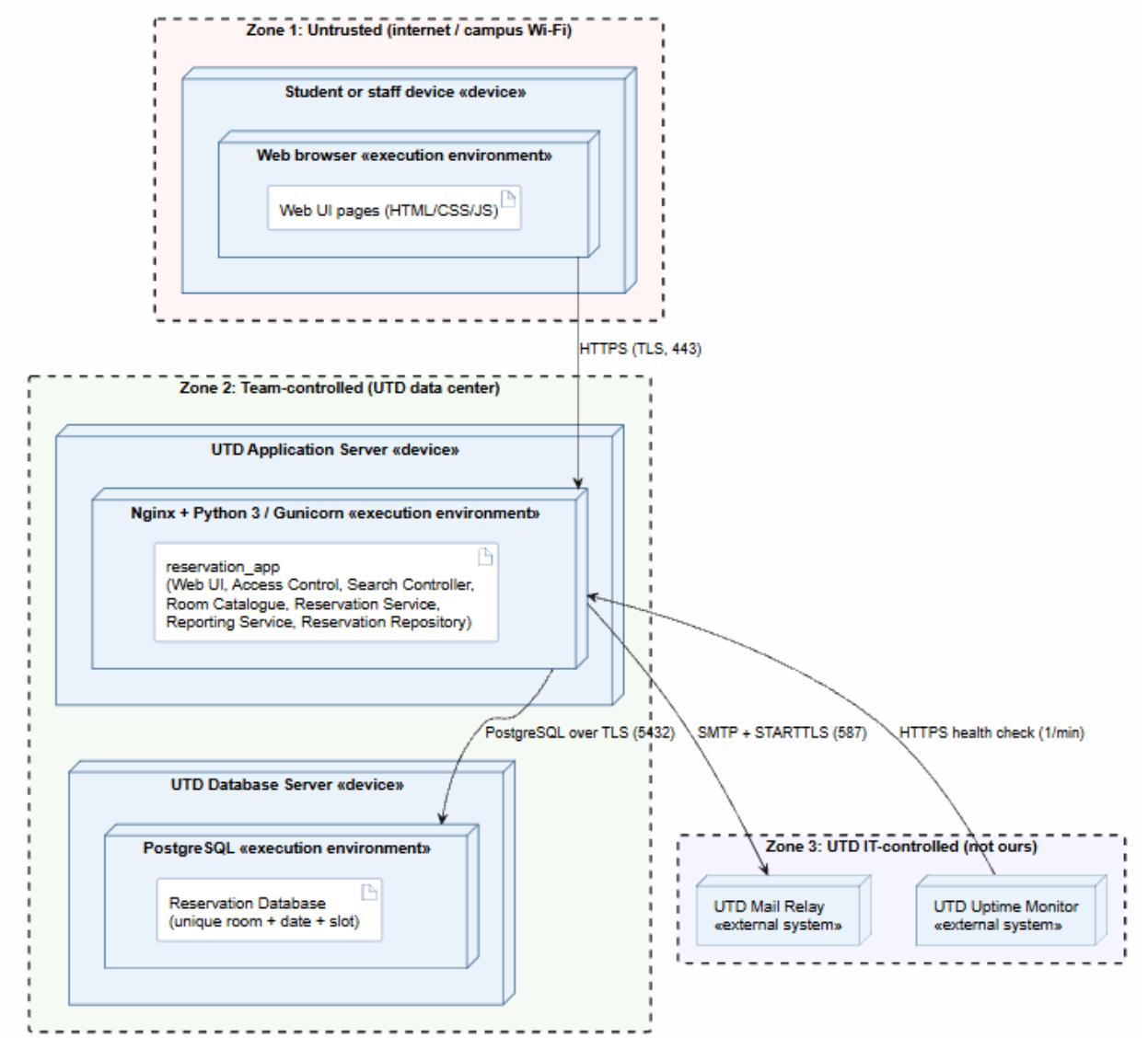


Figure 6.4 UML deployment diagram: nodes, execution environments, deployed artifacts, communication paths with protocols, trust boundaries

Node	What runs on it (artifacts, components)	Communicates with (protocol)	Trust zone
Student or staff device (web browser)	Web UI pages (HTML/CSS/JS) served by the app	UTD Application Server (HTTPS, TLS, port 443)	Zone 1: untrusted
UTD Application Server (Nginx + Python 3 / Gunicorn)	`reservation_app`: Web UI, Access Control, Search Controller, Room Catalogue, Reservation Service, Reporting Service, Reservation Repository	Browser (HTTPS 443); Database Server (PostgreSQL over TLS, 5432); UTD Mail Relay (SMTP + STARTTLS, 587)	Zone 2: team-controlled

Node	What runs on it (artifacts, components)	Communicates with (protocol)	Trust zone
UTD Database Server (PostgreSQL)	Reservation Database, with a unique constraint on room + date + slot	Application Server only (PostgreSQL over TLS, 5432)	Zone 2: team-controlled
UTD Mail Relay	UTD Email Service (external)	Application Server (SMTP + STARTTLS, 587)	Zone 3: UTD IT
UTD Uptime Monitor	External monitor that checks the home page every minute (QR-04)	Application Server (HTTPS health check)	Zone 3: UTD IT

6.5 From analysis to architecture

Analysis object (from 5.1 and 5.2)	Component that owns it (6.3)	Note (split, merged, external)
Student; Staff Member	Application Services (identity/role context)	Identity and role data are checked by the application; no separate identity integration is assumed in P2
Study Room; Room Listing Update	Room Catalogue	Room and listing-update concepts have persisted through the Reservation Database
Reservation; Availability Slot	Reservation Database	The database owns persistent reservation and slot state; Application Services applies reservation rules before committing (intentional split between behavior and persistence)

6.6 Design principles applied

Principle	Where it shows in your design	Evidence (component, interface, ADR)
Separation of concerns	Presentation, application/domain logic, and persistence are separate layers	Web UI, Application Services, Reservation Database; ADR-01
Information hiding	The UI and application services use catalogue and persistence interfaces rather than database details	Room Catalogue and Reservation Database interfaces; ADR-01

Principle	Where it shows in your design	Evidence (component, interface, ADR)
Low coupling	Email delivery is isolated behind the Notification Adapter and cannot reverse a committed reservation.	Notification Adapter; ADR-03
High cohesion	Reservation validation and conflict handling are kept in Application Services and the reservation persistence boundary.	Application Services + Reservation Database; ADR-02
Design for change	The email provider and database implementation are accessed through defined interfaces; actual UTD deployment remains a constraint.	Notification Adapter and persistence interface; ADR-03
Keep it simple (KISS)	A layered application is used instead of separate microservices or a message broker.	Section 6.2; ADR-01

7 Design Rationale

7.1 Architecture decision records

ADR-01 [Use a layered three-tier Architecture]

Status	Accepted
Date and owner	11/8/2026 – Team 07
Context (drivers)	The system needs to separate the user interface, reservation logic and data storage. QAS -01 requires fast room searches, QAS-02 requires reliable reservation handling, and QAS-03 requires making the reservation system easy for students to understand.
Options considered	1) Layered three-tier web application 2) Microservices architecture which splits the system into smaller services that can run independently 3) Event driven architecture: uses events to let different parts of the system to communicate with one another
Decision	Layered three-tier web application architecture
Consequences	The decision makes it easier to organize, understand and update. The downside is that the system will run on one main application, so individual parts cannot be scaled separately
Evidence	Section 6.2 compares the three architectural styles.
Related	QAS-01, QAS-0-2, QAS-03, CON-01, CON-02

ADR-02 [Use Database transactions to prevent overlapping reservations]

Status	Accepted
Date and owner	11/8/2026 – Team 07
Context (drivers)	Multiple students could try and reserve the same room and time slot at the same time. The system must prevent duplicate reservations and satisfy QR-02, which needs one successful reservation out of 20 simultaneous attempts.
Options considered	1) Check room availability in the application before saving a reservation 2) Use database transactions to coordinate reservation attempts and prevent conflicting bookings
Decision	Use database transactions to handle the reservation creation and prevent any overlap. The database will enforce consistency when multiple requests are made at the same time
Consequences	This decision will prevent double bookings and keep the reservation data accurate. The downside of that is that saving reservations would be more complicated and students would be rejected if someone books the room first

Evidence	Architecture spike S-01 tested 20 simultaneous reservation attempts across 100 trials using Python and SQLite. All 100 trials passed.
Related	QR-02, FR-08, FR-09, UC-02, S-01, CON-01

ADR-03 [Separate Email notifications from reservation processing]

Status	Accepted
Date and owner	11/8/2026 – Team 07
Context (drivers)	The system must send confirmation emails when a reservation activity happens. Email delivery depends on UTD email service which is out of the systems control. The architecture should allow a reservation to process even if a email delivery fails
Options considered	1) Have the reservation service communicate directly with the UTD email service 2) Use a separate notification adapter to handle communication with UTD email service
Decision	Use a separate notification adapter to handle email communication between the system and the UTD email service
Consequences	This decision makes it easier to update the email service without having to change the reservation system. The downside is that the system will have another component that it has to maintain
Evidence	Section 6.6 identifies the Notification adapter as the component that is responsible for isolating email delivery. No separate email integration spike is done yet
Related	UC-02, UC-03, CON-01, CON-02

7.2 Trade-off summary

Quality favored	Architectural move	Cost accepted	Recorded in
Simplicity, maintainability, cohesion	Layered three-tier application rather than microservices or event-driven services	No independent deployment or scaling of individual services	ADR-01
Integrity and concurrency correctness	Atomic transaction and persistence-level conflict protection	Transaction contention and database-specific implementation work	ADR-02
Availability and loose coupling	Best-effort email through Notification Adapter after reservation commit	Email may be delayed or undelivered even when reservation succeeds	ADR-03

8 Prototype Evidence: Architecture Spike

Spike ID and title	S-01 - Concurrent Reservation Conflict Prevention
Risk or decision it addresses	The Study Room Reservation System must prevent multiple students from successfully reserving the same room during the same time slot, even when reservation requests occur concurrently. This spike evaluates the feasibility of using database transactions to ensure reservation consistency.
Question (answerable yes or no)	Can a transaction-based reservation mechanism ensure that exactly one of 20 simultaneous reservation attempts succeed while the remaining 19 are rejected?
Set-up	A standalone prototype was developed using Python 3.14 and SQLite. The program simulated 20 concurrent students attempting to reserve the same study room for the same time slot. SQLite transactions were used to coordinate reservation operations and prevent conflicting reservations. The experiment was repeated for 100 independent trials.
How it was measured	Each trial recorded the number of successful reservations, rejected requests, database errors, stored reservations, and total execution time. Results were exported to results.csv.
Result (numbers)	100/100 trials passed (2,000 total reservation attempts). Exactly 100 reservations succeeded, and 1,900 conflicting requests were rejected. There were 0 database errors and 0 double-bookings. Every trial stored exactly one reservation. Mean trial duration: 146.86 ms. Maximum trial duration: 288.33 ms.
What it changed or confirmed	The spike confirmed that transaction-based reservation handling prevented double-booking in all 100 local trials, satisfying the concurrency-correctness outcome of QR-02 and supporting FR-09 and UC-02. It supports enforcing reservation conflicts at the database transaction level rather than relying only on application-level availability checks. The final architecture should retain this protection, with additional integration testing during P3.
Where to see it	Source code, execution instructions, and experimental results are available in the GitHub repository under /spikes/S-01/. Files include reservation_spike.py, README.md, and results.csv. Commit: 8fac0da. Pull Request: #1.

9 Traceability

9.1 Extended traceability matrix

Requirement	Use case	Analysis classes	Component(s)	QAS / ADR	Evidence
FR-01	UC-01, UC-05	Study Room	Room Catalogue	QAS-01	UC-01, UC-05; §6.3
FR-02	UC-01, UC-05	Study Room	Room Catalogue	QAS-01	UC-01, UC-05; §6.3
FR-03	UC-01, UC-05	Study Room	Room Catalogue	QAS-01	UC-01, UC-05; §6.3
FR-04	UC-01	Study Room, Reservation	Search Controller; Reservation Repository	QAS-01	UC-01; §6.3
FR-05	UC-01	Study Room, Reservation	Search Controller; Reservation Repository	QAS-01	UC-01; §6.3
FR-06	UC-01	Study Room	Search Controller; Web UI	QAS-01, QAS-03	UC-01; §6.3
FR-07	UC-01, UC-02	Study Room, Reservation	Search Controller; Reservation Service	QAS-01, QAS-02	UC-01, UC-02; §6.3
FR-08	UC-02	Student, Reservation, Study Room	Reservation Service; Reservation Repository	QAS-02, ADR-02	UC-02; S-01
FR-09	UC-02	Reservation, Study Room	Reservation Service; Reservation Repository; Reservation Database	QAS-02, ADR-02	S-01; 8fac0da
FR-10	UC-03	Student, Reservation	Reservation Service; Reservation Repository	ADR-02	UC-03; §6.3
FR-11	UC-04	Student, Reservation	Reservation Service; Reservation Repository	ADR-01	UC-04; §6.3
FR-12	UC-05	Staff Member, Study Room	Room Catalogue; Access Control	ADR-01	UC-05; §6.3
FR-13	UC-06	Study Room, Reservation	Reporting Service; Reservation Repository	ADR-01	UC-06; §6.3

FR-14	UC-02	Student, Reservation	Reservation Service; Notification Adapter	ADR-03	UC-02; §6.3; §7.1
FR-15	UC-03	Student, Reservation	Reservation Service; Notification Adapter	ADR-03	UC-03; §6.3; §7.1
QR-01	UC-01	Study Room, Reservation	Search Controller; Reservation Repository	QAS-01	§6.1; test pending
QR-02	UC-02	Reservation, Study Room	Reservation Service; Reservation Repository; Reservation Database	QAS-02, ADR-02	S-01; 8fac0da
QR-03	UC-01, UC-02	Student, Study Room, Reservation	Web UI	QAS-03, ADR-01	§6.1; test pending
QR-04	All online UCs	—	Web UI; Reservation Service; Reservation Repository	ADR-01	§6.3; monitoring pending
CON-01	All data-handling UCs	Student, Reservation	Web UI; Reservation Repository; Reservation Database	ADR-01, ADR-02	§6.3; §7.1
CON-02	All data-handling UCs	Student, Reservation	Access Control; Reservation Repository; Reservation Database	ADR-01, ADR-03	§6.3; §7.1

9.2 Gap analysis

Gap	Items	What you did about it
Requirements with no component	None identified in the matrix	Mapped all 19 requirements to components from Section 6.3, including the Reservation Repository and Reservation Database for concurrency of correctness.

Gap	Items	What you did about it
Components with no requirement, QAS or ADR	UTD Email Service (external supporting system)	Identified email delivery supporting UC-02, UC-03 and ADR-03. All internal components have a requirement, use case, QAS, or ADR connection.
Use cases not analyzed (and why that is acceptable)	UC-03, UC-04, UC-06.	Detailed analysis focused on UC-01, UC-02, and UC-05. The remaining use cases are documented in Section 3.4 and mapped to components in Section 6.3; detailed analysis is deferred.

10 Project Plan (updated; was P1 section 4)

10.1 Roles and working agreement

Team member	Role for P1 to P2	How the team makes a decision when it disagrees
Matthew Low	Requirements manager and repository maintainer	Discuss issues with the team through Discord voice call twice every week, compare alternatives against the agreed requirements and record the decision
Andrew Artiaga	Scope, system context, and repository coordinator	Discuss disagreements with the team through Discord voice call twice every week, compare options against the project scope and stakeholder needs and document decision
Arjun Sharma	Stakeholder and elicitation manager	Discuss disagreements with the team through Discord voice call twice every week, review/evaluate stakeholder needs and document decision
Ernie Hsiao	Non-functional, constraints, and prototyping analyst	Discuss disagreements with the team through Discord voice call twice every week, evaluate decisions against system performance and quality attributes, and document the decision
Artyom Gevorgyan	Verification and quality requirements analyst	Discuss disagreements with the team through Discord voice call twice every week, compare and analyze options for reaching quality requirements, and document the decision
Param Kathiravan	Use case analyst	Discuss disagreements with the team through Discord voice call twice every week, compare proposed solutions against functional requirements and use

		cases, and document the agreed decision.

10.2 Plan to P3

Milestone (note: maybe not in order)	Date	Owner	Done when
Build a baseplate for the code	11/8/2026	Matthew Low & Arjun Sharma	Nov 19
Test code and edge cases	11/8/2026	Matthew Low & Arjun Sharma	Nov 29
Document test and edge cases	11/8/2026	Andrew Artiaga & Param Kathiravan	Nov 29
Complete a final product for the code	11/8/2026	All team members	Dec 6
Complete the team review and resolve identified document issues	11/8/2026	All team members	Dec 7
Submit the final system, testing, report, and demo for P3	11/8/2026	All team members	Dec 9

10.3 Risks (updated)

Risk	Status since P1	Likelihood / impact	Mitigation and owner
R1 Student sign-in cannot be connected to UTD's identity service, so the system cannot confirm who is booking.	Unchanged This requires us to build our own system to recognize different students and will take time.	High / Medium	Arjun Sharma – Define a simple identification method that does not require UTD identity integration.
R2 Requirements change after this update and cause rework in P3.	Downgraded Requirements are unlikely to change at this point.	Low / High	Matthew Low – Record the requirement changes in the repository and assess the impact on scope and deadlines. Also make sure the team agrees to any changes proposed for the baseline
R3 A member is unavailable during	Unchanged	Medium / Medium	Ernie Hsiao – Have each member document their progress and upload their work to GitHub regularly. Share

exam season, and their work is delayed or missing.	Members should still communicate when delayed work is expected.		status on tasks with the team and reassign unfinished tasks if necessary
R4 Two students are able to reserve the same room and time under load.	Closed ADR-02 addresses this issue.	Low / High	Artyom Gevorgyan – Design a reservation conflict check that will recheck availability on reservations when submitted.
R5 Scope grows to include excluded items such as SMS, Duo, or door locks.	Downgraded Plans to include these would cause major changes towards the direction of the project. These changes are likely to cause delays.	Low / Medium	Andrew Artiaga – Keep a list of extra features whether they are used or not and decide with the team whether these features fit in the scope of the project or not.
R6 (new) Reservation information may be stolen or manipulated.	Security issue if we need to create our own identification method.	Medium / High	Ernie Hsiao - Define a simple encryption method that stores the user's identification.

10.4 Repository

https://github.com/matthewlow262-jpg/Study-Room-Reservation-System.git

Folder layout: /P2-Files, /P2-Files/Diagrams, /spikes/S-01

Commit conventions and Branch: We have decided to split the sections into branches and commit from there. We then pull requests for the sections and review each other's work through the document.

Access and permissions: Team members should have the repository permissions needed to access all the team's files.

How issues are used: We will mainly use GitHub to document our progress, and issues will be discussed by team members to come to a consensus.

10.5 Use of AI tools in P2

Member	Tool	What it was used for	What you checked or changed
Andrew Artiaga	Claude	Understanding component diagram	Components of 6.3 and reviewed Provides and Requires relationships

Member	Tool	What it was used for	What you checked or changed
Param Kathiravan	ChatGPT	Assisted with developing the S-01 Python/SQLite concurrency spike	Executed 100 trials locally, verified all results, and cross-checked requirements against Sections 6 and 7.

11 Open Questions (updated; was P1 section 5)

ID	Question	Raised by	Owner and due date	Status or answer	Assumption meanwhile
Q1	Will the system use actual UTD study room information or a simulated dataset?	Team members; UTD library staff	October 25, 2026	We will use a simulated dataset.	The system will use a simulated dataset if actual UTD room information is unavailable.
Q2	How will the students identify themselves when making, viewing, or canceling reservations?	Project team; UTD IT staff authentication is available	October 25, 2026	We will likely use UTD net IDs.	The initial version will use a simple project-level identification method and will not integrate with UTD Duo.
Q3	Will the system send email confirmations, and what email service can the team use?	Project team; the instructor or UTD IT staff	October 25, 2026	We will likely use email confirmations.	Email notifications are optional until the team confirms that an email service is an available option.

12 Baseline v1.0 Declaration (was P1 section 6)

We declare this document, version 1.0, the baseline for P2. Sections 1 to 9 describe what we will build in P3. Any later change to them is recorded in section 0.2 with its reason.

Team member	Signature or typed name	Date
Matthew Low	Matthew Low	11/8/2026
Artyom Gevorgyan	Artyom Gevorgyan	11/8/2026
Ernie Hsiao	Ernie Hsiao	11/8/2026
Arjun Sharma	Arjun Sharma	11/8/2026

Andrew Artiaga	Andrew Artiaga	11/8/2026
Param Kathiravan	Param Kathiravan	11/8/2026